Why LLM Outputs Degrade — And What Actually Combats It

2026-06-21 • 33 Sources

39%
AVG MULTI-TURN PERFORMANCE DROP

18/18
FRONTIER MODELS SHOW CONTEXT ROT

99.3%→69.7%
GPT-4O NOLIMA DROP BY 32K

6
DISTINCT DEGRADATION MECHANISMS

EXECUTIVE SUMMARY

Large language model output quality is not a fixed property of a model; it erodes predictably as a function of how the model is fed and how long it is asked to run. The research consensus across 2023–2026 identifies several mechanically distinct failure modes that practitioners often lump together as "the model got worse." The dominant one is *context rot*: Chroma's 2025 evaluation of 18 frontier models (including GPT-4.1, Claude Opus 4, and Gemini 2.5) found that every model degrades as input length grows, even on trivial tasks and well before the context window is full [1] [2]. This builds on the foundational "lost in the middle" finding that models follow a U-shaped attention curve, reliably using information at the start and end of a prompt while neglecting the middle [3].

Degradation also accumulates over time and turns, not just tokens. Models lose 39% of their performance on average when a task is delivered across multiple conversational turns rather than in one fully-specified prompt, and once they take a wrong turn they "get lost and do not recover" [4]. In long agentic runs, errors compound super-linearly: the per-step error rate itself rises as the model conditions on its own mistaken history, an effect that scaling model size does not fix [11]. Separately, at the decoding layer, maximization-based generation produces bland, repetitive degeneration unless the unreliable probability tail is truncated [5], and excessive "thinking" tokens in reasoning models cause measurable accuracy losses through overthinking [15][17].

The countermeasures converge on a single principle articulated by Anthropic: find "the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome" [6]. In

> practice this means context engineering (compaction, tool-result clearing, sub-agent isolation, just-in-time retrieval, external note-taking) [6][19], retrieval reordering to exploit the U-curve [14] [21], architectural fixes such as attention sinks [13], task decomposition with verification to contain error propagation [12][29], and prompt discipline that treats every token as a cost—especially for reasoning models, where shorter and clearer prompts outperform verbose scaffolding [26]. The unifying lesson: bigger context windows and longer reasoning are capacities, not guarantees, and quality is won by curation, not accumulation.

INTRODUCTION

The premise of this report is a question that has become central to anyone building on top of language models: why does the same model that answers brilliantly in a clean, short prompt produce sloppy, forgetful, or self-contradictory output in a long document analysis, a multi-hour coding session, or a sprawling chat? The marketing answer—"the context window is huge, just put everything in"—is contradicted by the empirical record. As one practitioner summary puts it, a million tokens is "a technical capacity, not a performance guarantee," with real-world models performing best at roughly 5,000–50,000 tokens and reliably missing information past 500,000 [25].

This report scopes "degradation" broadly and deliberately, because the term hides at least six distinct phenomena with different causes and different fixes. The first is *input-length degradation* (context rot): quality falls as the prompt grows, independent of task difficulty [1]. The second is *positional degradation*: information in the middle of a context is used less reliably than information at the edges [3]. The third is *temporal/conversational degradation*: quality erodes across dialogue turns and underspecified instructions [4]. The fourth is *error accumulation*: in long autoregressive runs, mistakes compound and become irreversible [11][12]. The fifth is *decoding-level degeneration*: the token-sampling process itself produces repetitive or incoherent text [5]. The sixth is *output-side overthinking*: reasoning models that generate too many tokens abandon correct answers [15]. The user's framing—context rot, unclear instructions, and excessive tokens—maps directly onto these mechanisms, and the report treats both the input side (what we feed the model) and the output side (what we let it generate) as sources of decay.

The methodology is documented fully in the appendix. In brief, this is a "deep" mode investigation drawing on 30 sources registered with stable identifiers, spanning foundational peer-reviewed papers (the 2019 nucleus-sampling work [5], the 2023 lost-in-the-middle study [3]), rigorous 2024–2025 benchmarks (RULER [9], NoLiMa [10]), recent 2025–2026 mechanistic and agentic analyses [11][12][24][29], and primary industry engineering guidance from Anthropic and others [6][19]. Where a claim rests on a single source or anecdote, it is flagged. Two assumptions are surfaced here because they shape the analysis: first, that findings on frontier models in 2025–2026 generalize to the models most readers use (supported by the breadth of Chroma's 18-model sweep [1]); and second, that "output quality" is meaningfully measurable through the task-accuracy and text-quality proxies these studies employ, even though no

single metric captures it. The report is organized from symptoms to mechanisms to mitigations, closing with synthesis, limitations, and concrete recommendations.

MAIN ANALYSIS

The analysis proceeds from symptoms to mechanisms to mitigations across eleven findings: context rot (Finding 1), positional degradation (2), the claimed-vs-effective context gap (3), the attention-budget mechanism (4), conversational degradation (5), the four context failure modes (6), error accumulation (7), decoding-level degeneration (8), excessive-token effects (9), context engineering as the master mitigation (10), and targeted technical interventions (11).

Finding 1: Context Rot — Quality Falls as Input Grows, Long Before the Window Is Full

The single most consequential and best-documented form of degradation is what Chroma's 2025 research formalized as "context rot": the measurable decline in output quality as input length increases, even when the context window is nowhere near full [1][2]. The study's design is what makes it persuasive. Long-context benchmarks are notoriously confounded because longer inputs usually mean harder tasks, so a quality drop could reflect difficulty rather than length. Chroma controlled for this by holding task complexity constant while varying only input length, allowing them to isolate the effect of length alone [1]. Under these minimal conditions, "model performance degrades as input length increases, often in surprising and non-uniform ways," and they note that real applications are more complex, "implying that the influence of input length may be even more pronounced in practice" [1]. Crucially, the effect was universal: across 18 frontier models including GPT-4.1, Claude Opus 4, Gemini 2.5, and Qwen3, "every single one gets worse as input length increases. Not some. Not most. All of them" [2].

The degradation is non-uniform in instructive ways. Chroma varied the semantic similarity between the "needle" (the target fact) and the question, quantified by embedding cosine similarity, and found that "as needle-question similarity decreases, model performance degrades more significantly with increasing input length" [1]. This matters enormously for real systems, because exact lexical matches between a user's question and the answer text are rare; most production retrieval involves semantic rather than literal matching, exactly the regime where rot is worst [1][2]. The study also showed that distractor content (plausible but irrelevant passages) and the structure of the "haystack" both modulate the decline, and that on a conversational long-memory benchmark (LongMemEval) there was a substantial accuracy gap between a focused ~300-token input and the same information embedded in ~113K tokens [2]. The headline implication is counterintuitive but now well-supported: the same information yields radically different performance depending on how much surrounding context it is buried in.

Industry analysis has translated this into operational heuristics. Multiple practitioners converge on a working model in which contexts up to roughly 50,000 tokens are generally safe, 50,000–200,000 tokens introduce quality dips, and beyond 200,000 tokens models frequently underperform shorter contexts despite the window technically supporting far more [25]. A widely cited engineering observation from the

SWE-rebench maintainers describes a "1M token wall" where performance degrades meaningfully past roughly a million tokens regardless of advertised window size [2]. Salesforce frames the same phenomenon in reasoning terms: "modern models' reasoning capabilities degrade long before they hit their stated context window limits," with accuracy on modified clinical-reasoning evaluations dropping by more than a third once questions penalized surface pattern-matching [18]. The practical upshot, repeated across sources, is that a 30,000-token context of curated content typically beats a 300,000-token context with everything dumped in—on quality, cost, and latency simultaneously [25].

Finding 2: Positional Degradation — The "Lost in the Middle" U-Curve

If context rot describes how much you put in, the "lost in the middle" phenomenon describes where you put it. The foundational study by Liu et al. (2023) analyzed model performance on multi-document question answering and key-value retrieval while systematically varying the position of the relevant information. Their central result is now one of the most replicated findings in applied LLM research: "performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts" [3]. The degradation holds "even for explicitly long-context models," and performance "substantially decreases as the input context grows longer" [3]—directly linking positional bias to context rot.

The resulting accuracy profile is a distinctive U-shaped curve, driven by two biases that push information out of the middle. *Primacy bias* gives disproportionate attention to the earliest tokens—the system prompt, the top of a document—partly because models develop strong "attention sinks" on early tokens during training [3]. *Recency bias* favors the most recent tokens, because next-token prediction is trained to lean heavily on what immediately precedes the prediction point, which is why placing the actual question last works so well in practice. In the most striking version of the result, models in the middle-position condition sometimes underperformed a *closed-book baseline*—meaning the long context actively hurt them relative to having no documents at all [3]. The bias appears across model families and sizes; encoder-decoder architectures are somewhat more position-robust but only up to the lengths they were trained on, after which the same U-shape re-emerges [3].

The mechanism connects positional degradation to architecture, and this is best understood as synthesis across the structural findings. Most modern transformers use rotary position embeddings (RoPE), which encode token position by rotating query and key vectors. When a model trained on, for example, 8K tokens is stretched to 128K or 1M via position interpolation or YaRN, those rotations are squeezed into ranges the model barely encountered in training, blurring long-range positions—and that blur lands hardest on the middle, far from the strongly anchored start and the recency-favored end. Liu et al. attribute the effect to how language models "encode positional information and manage attention" and confirm it persists in explicitly long-context models [3]. The lost-in-the-middle effect is therefore not a bug to be patched but a structural property of how attention and positional encoding interact, which is why, as the next finding details, the most reliable fixes manipulate *placement* rather than hoping the model attends uniformly.

Finding 3: The Claimed-vs-Effective Context Gap, Quantified

The marketing number on a model card ("128K context," "1M context") and the length at which the model actually performs well are two different quantities, and two rigorous benchmarks have measured the gap. NVIDIA's RULER (2024) generates synthetic tasks across four categories—retrieval, multi-hop tracing, aggregation, and question answering—with configurable length and complexity, then benchmarks models against a quality threshold defined as Llama-2-7B's performance at 4K tokens (85.6%) [9]. The results are sobering: "despite achieving nearly perfect performance on the vanilla needle-in-a-haystack test, almost all models exhibit large degradation on more complex tasks in RULER as sequence length increases" [9]. Concretely, "while all models claim context size of 32K tokens or greater, only half of them can effectively handle sequence length of 32K," and "almost all models fall below the threshold before reaching the claimed context lengths" [9]. Even the strongest model at the time (GPT-4) showed non-marginal degradation of about 15 points extending from 4K to 128K [9]. RULER also showed that non-transformer architectures (RWKV, Mamba) degraded sharply beyond 8K, underperforming the transformer baseline [9].

NoLiMa (Modarressi et al., 2025) tightened the screws by attacking the single biggest weakness of needle-in-a-haystack tests: literal lexical overlap. Standard NIAH lets a model "cheat" by string-matching the question to the needle, so NoLiMa designed a needle set with "minimal lexical overlap, requiring models to infer latent associations to locate the needle within the haystack" [10]. Under this more realistic condition, the decline is severe: across 13 models claiming at least 128K context, performance was strong below 1K tokens but "at 32K, 11 models drop below 50% of their strong short-length baselines." Even GPT-4o, one of the best performers, fell "from an almost-perfect baseline of 99.3% to 69.7%" [10]. NoLiMa defines a model's "effective length" as the longest context at which it retains at least 85% of its base score, and for most models this is far below the advertised window [10]. The authors attribute the decline to "the increased difficulty the attention mechanism faces in longer contexts when literal matches are absent," and note pointedly that "even models enhanced with reasoning capabilities or CoT prompting struggle to maintain performance in long contexts" [10].

Taken together, RULER and NoLiMa convert the qualitative claim of context rot into hard numbers and establish a vocabulary—*claimed length* versus *effective length*—that every team should adopt when choosing a model. The convergence is what gives the finding weight: an industry lab studying frontier models on minimal tasks [1], an academic synthetic-task benchmark [9], and an academic association-based benchmark [10] all reach the same conclusion through independent methods. The practical recommendation that follows is to benchmark position robustness and effective length on representative tasks rather than trusting the advertised window, and to treat any context beyond a model's measured effective length as a region where critical facts may be silently dropped.

Finding 4: The Mechanism — A Finite Attention Budget and Its Sinks

Why does any of this happen? The most coherent mechanistic account, echoed from architecture papers to vendor documentation, is that the transformer's attention operates under a finite budget. Salesforce states it directly: "the transformer's attention mechanism distributes a finite attention budget across every token in the input. As the input grows, the weight assigned to any single instruction decreases. This

dilution allows irrelevant data to distract the model, leading to logical errors and hallucinations" [18]. Anthropic frames context engineering around exactly this constraint, describing the model as having a "finite attention budget" that good engineering must spend wisely [6]. The deepset/Haystack analysis adds the competitive dimension: "irrelevant or redundant content doesn't just waste space—it actively competes with the information that actually matters. A critical instruction buried under pages of tool outputs may receive less attention than if it had been sent alone," and cites Anthropic's finding that models "remain capable at longer contexts but show reduced precision for information retrieval and long-range reasoning compared to shorter ones" [20].

The "attention sink" phenomenon, identified by Xiao et al. (2023) in their StreamingLLM work, supplies the missing micro-mechanism for primacy bias. The authors discovered that models allocate "a surprisingly large amount of attention score to the initial tokens, irrespective of their relevance to the language modeling task," and attribute this to the mathematics of the softmax operation, which must distribute attention weight somewhere even when no token is truly relevant [13]. They demonstrated that naive sliding-window attention "fails when the text length surpasses the cache size" because it evicts these initial sink tokens—and that simply "keeping the KV of initial tokens will largely recover the performance of window attention," with "just 4 initial tokens" sufficing to stabilize attention over up to 4 million tokens [13]. This is both a mechanistic insight (the start of the context is load-bearing for the entire attention distribution) and, as discussed later, a practical mitigation.

The mechanism explains the otherwise puzzling pattern of symptoms. Dilution explains why instructions get silently ignored as context grows [18]; sink behavior and recency explain the U-curve [3][13]; and the competition for a fixed budget explains why adding *irrelevant* content degrades performance even on tasks that do not need it [20]. It also clarifies why the problem is not solved by larger windows: expanding the window enlarges the denominator over which attention is spread without increasing the total budget, so "more capacity without better curation just means more room for noise to crowd out critical information" [18]. This is the single most important conceptual takeaway of the report, and every mitigation in the later sections is, at root, an attempt to protect a scarce attention budget from being squandered.

Finding 5: Conversational Degradation — Getting "Lost" Over Multiple Turns

Degradation is not only a function of token count; it is a function of *interaction structure*. Laban et al. (2025), in work accepted as an ICLR 2026 oral, ran over 200,000 simulated conversations to compare single-turn, fully-specified prompts against the same task delivered piecemeal across multiple turns ("sharded"). The result is one of the most actionable findings in the field: "all the top open- and closed-weight LLMs we test exhibit significantly lower performance in multi-turn conversations than single-turn, with an average drop of 39% across six generation tasks" [4]. Even top-tier models were not spared—Claude 3.7 Sonnet, Gemini 2.5 Pro, and GPT-4.1 each lost 30–40% in the sharded condition—and reasoning models such as o3 and DeepSeek-R1 "degrade just like their non-reasoning counterparts," because their longer responses simply give more room for incorrect assumptions [4][22].

The study's most important contribution is its decomposition of the loss into two components: a minor decline in *aptitude* (peak capability) and a large increase in *unreliability* (variance) [4]. In single-turn settings, more capable models tend to be more reliable; in multi-turn settings, "all LLMs exhibit very high unreliability regardless of aptitude" [4]. The behavioral story behind the numbers is vivid: "LLMs often make assumptions in early turns and prematurely attempt to generate final solutions, on which they overly rely," and "when LLMs take a wrong turn in a conversation, they get lost and do not recover" [4]. A control condition is decisive for diagnosis. When all the sharded pieces were simply concatenated back together (CONCAT), performance recovered to about 95% of the fully-specified baseline—proving that "information loss from sharding isn't the reason why performance drops" [22]. The damage comes from the model committing early to a flawed interpretation and then conditioning all subsequent generation on its own premature answer, which remains in the context polluting later reasoning.

A 2026 follow-up reframes the root cause as an *intent alignment gap* rather than a capability deficit. The authors argue that "Lost in Conversation" arises "from structural ambiguity in conversational context rather than representational limitations," and prove theoretically that "scaling model size or improving training alone cannot resolve this gap" [23]. Their proposed fix—a Mediator-Assistant architecture that rewrites vague user inputs into explicit, well-structured instructions before the assistant executes—substantially recovers multi-turn performance [23]. This finding directly validates the user's intuition that "unclear instructions" degrade output: underspecification in early turns is precisely the trigger, and the practical mitigation (covered later) is either to specify fully up front or to periodically consolidate the conversation into a clean, complete restatement of the task.

Finding 6: The Four Failure Modes of Accumulated Context

Drew Breunig's widely-cited 2025 taxonomy, popularized further by Simon Willison and republished by O'Reilly, gives the field a shared vocabulary for *how* an overloaded context degrades output, distinguishing four mechanistically different failure modes [7][8][27]. The taxonomy is valuable precisely because the four modes have different triggers and different remedies, and conflating them leads to the wrong fix.

Context poisoning occurs "when a hallucination or other error makes it into the context, where it is repeatedly referenced" [7]. Its danger is persistence: once an error is recorded in the context, it is reinforced by every subsequent reference, "creating a spiral away from reality" that ordinary prompting cannot easily undo [7][8]. In agentic systems this is acute, because a single hallucinated fact written to a scratchpad or plan can corrupt every downstream step. *Context distraction* occurs "when a context grows so long that the model over-focuses on the context, neglecting what it learned during training" [7]. The canonical illustration comes from the Gemini-based Pokémon-playing agent, whose builders observed that beyond roughly 100,000 tokens the agent "showed a tendency toward repeating actions from its vast history rather than synthesizing novel plans" [7]. Practitioners report that smaller models hit this distraction ceiling far earlier, around 32,000 tokens [7].

Context confusion occurs "when superfluous content in the context is used by the model to generate a low-quality response" [7]. The clearest evidence is from function-calling: the "Less is More" work cited by Breunig showed that Llama 3.1 8B fails a tool-use benchmark when given 46 tools but succeeds with only 19, because the model "incorporates everything in context into their probability distribution"—unlike a human, it cannot simply ignore the irrelevant tools [7][8]. *Context clash* occurs "when you accrue new information and tools in your context that conflict with other information in the prompt" [7]. Breunig explicitly connects this to the multi-turn finding: the 39% sharded-prompt degradation is partly a clash phenomenon, where "early attempts by the model to answer the challenge before it has all the information remain present in the context and influence the model when it generates its final answer" [7][8]. The four modes are not independent—poisoned context can later trigger clash, and distraction worsens as confusion accumulates—but naming them separately is what makes targeted mitigation possible [8].

Finding 7: Error Accumulation and Self-Conditioning in Long-Horizon Runs

In long agentic tasks—coding sessions, multi-step tool use, formal reasoning—a distinct and arguably more dangerous form of degradation appears: errors do not merely persist, they *compound*, and the rate at which the model errs actually increases as the task proceeds. The cleanest demonstration is "The Illusion of Diminishing Returns" (2025), which separated long-context degradation from a newly named effect, *self-conditioning*. The naive assumption is that long-task failure is just a constant per-step error rate compounding over many steps. The authors show this is wrong: "the per-step error rate itself rises as the task progresses" [11]. The cause is mechanistic and tied to the autoregressive objective: "as a significant fraction of model training is to predict the most likely next token given its context, conditioning models on their own error-prone history increases the likelihood of future errors" [11]. They verified this by injecting errors into the model's visible history and watching subsequent accuracy "sharply degrade" as the injected error rate rose [11]. Most importantly for system designers, this effect "is not mitigated by scaling model size"—it is orthogonal to the long-context problem and will not be solved by a bigger or smarter model alone [11].

Theory corroborates the empirics. "Intrinsic Stability Limits of Autoregressive Reasoning" (2026) proves, under mild assumptions, that "the decision advantage of autoregressive reasoning trajectories decays exponentially with execution length," establishing that instability "can emerge as an inherent property of autoregressive execution dynamics rather than merely an artifact of specific architectures" [12]. The paper introduces the notion of an intrinsic "critical length" beyond which a single unbroken reasoning chain becomes unreliable, and concludes that "stable long-horizon reasoning requires structural segmentation and intermediate stabilization mechanisms," motivating a shift "from purely linear execution toward structured graph-like organization" [12]. A planning-centric analysis reaches a complementary conclusion: under reasoning-based policies, "errors under reasoning-based policy are effectively irreversible," with trajectories deviating "within the first few decisions" and rarely recovering thereafter, and only future-aware lookahead "both postpones the first error and substantially improves recovery" [31]. A 2026 FMEA-style taxonomy of agentic failure catalogs "history error accumulation" and "compounding mistakes carried across prior steps" as a core process-level risk, noting that "even a single undetected mismatch

early on can cascade into repeated invalid actions" with recovery becoming "progressively harder" over longer trajectories [24].

There is, however, a sharp tension in the mitigation literature worth flagging. Extreme task decomposition —executing every step in complete isolation to eliminate context dependence—solves error accumulation but introduces a new failure: the "no-recovery bottleneck." LEAD (2026) shows that "its memoryless design makes local errors irreversible," and because errors concentrate on a few "hard" steps, "once a model becomes consistently wrong on even a single step, success becomes statistically impossible—a bottleneck that persists even with majority voting" [29]. The lesson is that neither monolithic long chains nor fully atomized isolation is sufficient; robust long-horizon execution requires segmentation *plus* an error-recovery mechanism, a design point the recommendations return to.

Finding 8: Decoding-Level Degeneration — Repetition and the Unreliable Tail

The forms of degradation above concern *what the model attends to*; a separate, older problem concerns *how tokens are sampled* once the model decides what to say. Holtzman et al.'s 2019 "The Curious Case of Neural Text Degeneration" identified the counterintuitive core issue: although likelihood is an excellent training objective, using it as a *decoding* objective backfires. "Maximization-based decoding methods such as beam search lead to degeneration—output text that is bland, incoherent, or gets stuck in repetitive loops" [5]. The diagnosis was distributional: the probability distributions of strong language models "have an unreliable tail which needs to be truncated during generation," and simple fixes like fixed top-k or temperature scaling fail to suppress that tail adequately [5].

Their solution, nucleus (top-p) sampling, has become the default decoding strategy precisely because it addresses the mechanism directly. Rather than a fixed candidate count, it samples from "the dynamic nucleus of tokens containing the vast majority of the probability mass"—the smallest set whose cumulative probability exceeds a threshold p—so the candidate pool "expands and contracts dynamically" depending on the model's confidence [5]. When the model is confident, the nucleus is small and output stays coherent; when uncertain, it widens to permit human-like diversity. Human evaluation confirmed the resulting text was both higher quality and as diverse as human writing, where maximization produced repetition and pure sampling produced incoherence [5].

This failure mode is less discussed in the 2025–2026 agentic context but remains foundational and operationally relevant. Repetition loops, degenerate "stuck" output, and bland filler are decoding pathologies, not context pathologies, and they are tuned away at the sampling layer (temperature, top-p, repetition penalties) rather than through context engineering. It is worth noting the honest caveat that even top-p sampling "is not without shortcomings" and "can produce text with undesirable repetitions" in some regimes [5]. The practical implication is that degradation diagnosis must distinguish layers: a model producing repetitive or incoherent *text* likely has a decoding-parameter problem, while a model producing fluent but *wrong or forgetful* content likely has a context or attention problem—two different diseases requiring two different treatments.

Finding 9: Excessive Tokens — Verbose Prompts, Bloated Reasoning, and Overthinking

The user's third framing—"excessive tokens"—turns out to name two related but distinct pathologies, one on the input side and one on the output side. On the input side, verbose or over-engineered prompts consume the budget that should be reserved for reasoning and degrade output, an effect that is especially pronounced for reasoning models. Practitioner guidance grounded in DeepSeek's own documentation is blunt: reasoning models "already think step by step," so adding "think step by step" or chain-of-thought scaffolding "duplicates the behavior into the visible output and crowds out the actual answer," and few-shot examples act as unwanted constraints—"every example you remove from a reasoning-model prompt is one fewer constraint on the model's internal reasoning" [26]. DeepSeek explicitly warns that for R1, "few-shot prompting consistently degrades its performance," recommending zero-shot problem statements instead, and states that "too much information reduces accuracy" [26]. One engineering guide reports that reasoning quality degrades around the 3,000-token instruction mark and documents a concrete case where trimming a continuation prompt from roughly 4,200 to 2,200 tokens (a 47% reduction) "eliminated unnecessary phase splits and improved plan quality—with zero loss of instruction compliance" [32]. The mechanism is the same finite attention budget: every instruction token competes with every other, so an unclear or padded prompt literally dilutes the signal of the instruction that matters.

On the output side, the rise of reasoning models created a new failure mode: *overthinking*, where generating too many reasoning tokens actively reduces accuracy. This is now documented across multiple independent studies. "When More Thinking Hurts" (2026) finds that "marginal returns diminish substantially at higher budgets and that models exhibit overthinking, where extended reasoning is associated with abandoning previously correct answers," with marginal utility turning negative beyond roughly 12K tokens in one setup [15]. "Does Thinking More always Help?" (2025) reports a non-monotonic curve: "extending thinking at test-time initially boosts model accuracy, but performance degrades subsequently with prolonged thinking," and argues the early gains are "often illusory, stemming from increased response variance rather than genuine reasoning improvements" [16]. A saturation study quantifies the cliff at small scales: "the 1.5B model loses 2.4% accuracy when increasing from 512 to 1024 tokens," and "beyond saturation points, additional tokens can reduce accuracy" [17]. A complementary study of long reasoning models documents that they "generate redundant solutions that contribute minimally to accuracy and diversity, thereby wasting computational resources on simple problems," and shows that self-training to shorten reasoning reduces overhead "without compromising accuracy" across GSM8K, MATH500, GPQA, and AIME [30]. The mechanism overlaps with self-conditioning—a model that reaches a correct answer and keeps generating can drift off the correct trajectory through accumulated logical drift rather than arithmetic error [15][16].

The convergent recommendation across this literature is that test-time compute should be allocated adaptively, not maximally. Stopping at moderate budgets "can reduce computation significantly while maintaining comparable accuracy" [15], optimal thinking length varies with problem difficulty so "uniform compute allocation is suboptimal" [15], and distributing a fixed budget across multiple shorter reasoning

paths with majority voting ("parallel thinking") yields "up to 22% higher accuracy compared to sequential scaling" [16]. In short, both halves of the "excessive tokens" problem point the same way: tokens are not free, whether they sit in the prompt or in the model's own reasoning trace, and disciplined economy beats brute-force length.

Finding 10: The Master Mitigation — Context Engineering

If the unifying cause of degradation is a squandered attention budget, the unifying cure is *context engineering*, which Anthropic defines as "the natural progression of prompt engineering": where prompt engineering optimizes the wording of instructions, context engineering optimizes "the set of strategies for curating and maintaining the optimal set of tokens (information) during LLM inference" [6]. The guiding principle is the report's recurring refrain: "find the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome" [6]. This reframes the engineer's job from "how do I fit everything in the window" to "what is the minimal context that lets the model take the correct next action," a deliberately information-theoretic stance [6]. Several concrete techniques operationalize it, and notably each maps onto a specific failure mode from the earlier findings.

Compaction directly counters context rot and distraction by replacing accumulated history with a condensed summary when the context approaches a threshold. Anthropic's API documentation explains the rationale beyond mere token limits: "as conversations get longer, models struggle to maintain focus across the full history. Compaction keeps the active context focused and performant by replacing stale content with concise summaries" [19]. Their server-side implementation triggers by default around 150,000 tokens (configurable, minimum 50,000) and automatically drops superseded message blocks [19]. Effective compaction is selective—it must "preserve architectural decisions, unresolved bugs, and goal states while discarding redundant tool outputs and conversational filler"—and there is a documented hazard called "thrashing," where over-aggressive summarization drops information the agent later needs, forcing it to re-read files or re-run tools [20][28]. *Tool-result clearing* is the lightest-touch variant: once a tool result is deep in history, the raw payload is cleared while the message structure remains, recovering space without erasing the reasoning record [28]. Both are now first-party features on the Claude Developer Platform [19][28].

Sub-agent isolation (Breunig's "context quarantine") counters confusion and clash by giving each unit of work its own clean context window. As deepset describes it, "instead of one agent accumulating the full history of a complex task, you split the work across specialised subagents—each one receiving only the slice of context relevant to its job," while "the orchestrator maintains a thin, high-level context" and workers return only concise summaries [20][8]. *Just-in-time retrieval* counters confusion by loading data only when a step needs it rather than front-loading entire repositories—Anthropic's recommended approach for long-running agents, where the agent "maintains lightweight references and dynamically loads data at runtime" [6]. *Structured note-taking / context offloading* counters error accumulation and distraction by persisting state outside the window: agents maintain a scratchpad or progress file (Breunig's "context offloading," the `plan.md`/`NOTES.md` pattern, Claude Code's `claude-progress` memory) so long-horizon state survives even compaction [8][6][20]. A practitioner benchmark of five agent

configurations confirms these are not interchangeable; each technique "operates to make the context window more efficient in different ways," and the engineering skill is mapping each lever to the part of the workload it actually helps [28]. There is one important second-order caveat: compaction "can reduce context size but can also break prompt-cache reuse," so it should be done "at explicit boundaries" with a stable summary rather than re-summarizing the entire session every turn [33].

Finding 11: Targeted Technical Interventions — Retrieval, Architecture, Decoding, and Decomposition

Beyond the general discipline of context engineering, the literature supplies several precise, mechanism-matched interventions. The most cost-effective for retrieval-augmented systems is *retrieval reordering*, which weaponizes the lost-in-the-middle U-curve instead of fighting it. The recommended pipeline retrieves a larger candidate set (e.g., top-20), passes it through a cross-encoder *reranker* that scores each document against the query for higher precision, selects the top few, and then deliberately places "the most relevant document at the beginning and the second-most relevant at the end of the prompt," positioning weaker passages in the middle where attention is lowest [21][14]. The academic backing is direct: the "Long-Context LLMs Meet RAG" work proposes "reordering retrieved documents based on their retrieval scores… prioritizing documents with higher scores at the beginning and end of the input sequences to guide the LLMs' attention towards more relevant information and mitigate the impact of hard negatives" [14]. Pinecone frames the broader two-stage principle as "maximize retrieval recall by retrieving plenty of documents and then maximize LLM recall by minimizing the number of documents that make it to the LLM" via reranking—directly attacking context confusion by keeping noise out of the window [21]. The same paper also proposes training-based fixes (robustness fine-tuning on noisy retrieved context, and explicit relevance fine-tuning where the model is trained to identify relevant documents before answering) for teams that can fine-tune [14].

At the *architecture and decoding* layers, the interventions are different in kind. StreamingLLM's attention-sink fix—retaining the first few tokens' key-value states alongside a sliding window—stabilizes generation over millions of tokens without fine-tuning and yields up to a 22.2x speedup over recomputation, and the authors further show that pre-training with a dedicated sink token improves streaming behavior [13]. This is a degradation fix baked into the serving stack rather than the prompt. At the decoding layer, nucleus sampling and its relatives (temperature, repetition penalties) remain the standard remedy for degeneration, truncating the unreliable probability tail to prevent repetitive loops while preserving diversity [5]. These layers are largely invisible to application developers but are where vendors fight degradation on the user's behalf.

For *long-horizon agentic* degradation, the convergent prescription is decomposition plus verification plus recovery. The structural argument is that long single chains are intrinsically unstable, so work should be segmented—organized as a graph or DAG of atomic sub-tasks with "intermediate stabilization mechanisms" and resets that "shift the effective failure scale from the global reasoning horizon to local segment lengths" [12]. But because pure atomization creates the no-recovery bottleneck, segmentation must be paired with closed-loop verification (adversarial checkers, symbolic validators, or lookahead

evaluation that "substantially improves recovery across all stages") and, where affordable, redundancy via consensus voting [31][29][12]. For the conversational case, the matching mitigations are to consolidate underspecified multi-turn exchanges into a single well-formed instruction (the CONCAT result shows this recovers ~95% of single-turn performance [22]) or to insert an intent-clarifying rewrite step before execution [23]. The throughline across all of Finding 11 is that effective mitigation is mechanism-specific: reranking for positional bias, sinks for streaming stability, nucleus sampling for decoding degeneration, decomposition-plus-recovery for error accumulation, and instruction consolidation for conversational drift —there is no single universal patch.

SYNTHESIS & INSIGHTS

Stepping back from the individual findings, three cross-cutting insights emerge that are larger than any single study. The first is that **"degradation" is an umbrella term for at least six mechanically distinct diseases, and treating them as one leads to the wrong medicine.** A model that repeats itself (decoding degeneration [5]) needs different treatment than one that forgets a fact buried at token 60,000 (positional rot [3]), which needs different treatment than one that commits early to a wrong interpretation and never recovers (conversational drift [4]), which differs again from one whose error rate climbs as an agentic task lengthens (self-conditioning [11]). The most common practitioner mistake is to respond to all of these by adding more to the prompt—more instructions, more examples, more context—when the evidence says that for most of them, *adding* is the cause and *subtracting* is the cure.

The second insight is the **conservation-of-attention principle that unifies the input-side findings.** Context rot [1], lost-in-the-middle [3], context confusion [7], distraction [7], and verbose-prompt degradation [26] are all expressions of one fact: attention is a finite budget spread across all tokens, so every token added dilutes every other token's share [18][6]. This is why the benchmark gap between claimed and effective context exists [9][10], why irrelevant content hurts even when unused [20], and why a 30K curated context beats a 300K dumped one [25]. The corollary—the report's central, repeatable thesis—is that **bigger windows and longer reasoning are capacities, not guarantees.** Vendors sell the capacity; quality comes from the engineering that decides what to spend it on.

The third and most forward-looking insight is that **the field is converging on the same answer from radically different starting points.** A 2019 decoding paper [5], a 2023 positional study [3], 2024–2025 benchmarks [9][10], 2025–2026 mechanistic theory [11][12], and 2025 industry engineering guidance [6] [19] were produced by different communities with different methods, yet they all arrive at "minimal high-signal context, segmented work, and adaptive rather than maximal token budgets." When independent lines of inquiry converge like this, the conclusion is robust. A non-obvious second-order implication follows: as models get larger and windows get bigger, several of these problems do *not* go away—self-conditioning is explicitly shown to be unmitigated by scale [11], the intent-alignment gap is proven unsolvable by scale alone [23], and intrinsic autoregressive instability is architecture-general [12]. This means context engineering is not a temporary scaffold we will discard once models improve; it is a

durable discipline, and the teams that internalize it now are building skills that compound rather than expire.

LIMITATIONS & CAVEATS

Several limitations bound the confidence of this report. First, **measurement of "output quality" is heterogeneous.** The studies cited use different proxies—task accuracy on synthetic retrieval [9][10], multi-document QA [3], generation-task scoring [4], human judgments of text quality [5], and step-accuracy in long-horizon tasks [11]. These do not all measure the same thing, and a model can improve on one while degrading on another, so cross-study quantitative comparisons (e.g., "39% drop" versus "below 50% at 32K") should be read as directionally consistent rather than commensurable.

Second, **the frontier moves fast and some sources are non-peer-reviewed.** Several of the most recent and relevant items are preprints or vendor and practitioner publications (Chroma's report [1], Anthropic's engineering blog [6], Breunig's taxonomy [7][8], and various 2026 arXiv preprints [12][24][29][31]). These are included because they are the primary, original sources for the concepts in question and because the core claims triangulate across independent authors, but preprints have not completed peer review and vendor content carries an interest in promoting the vendor's tooling. Where a claim rested on a single anecdote—such as the Pokémon agent's 100K-token distraction ceiling [7]—it is labeled as anecdotal. Third, **the mitigation evidence is thinner than the diagnosis evidence.** Context rot and lost-in-the-middle are extensively benchmarked; the relative effectiveness of compaction versus sub-agents versus offloading is supported more by individual benchmarks [28] and engineering experience than by large comparative studies, and the field lacks a standard benchmark that scores mitigation strategies head-to-head.

Fourth, **model-specific behavior varies and generalization is imperfect.** Even within the universal trend, models differ: coverage of Chroma's study noted Claude models "decay the slowest" while sometimes refusing long tasks, whereas Gemini degrades earlier with higher variance and GPT models fail more erratically [1][2], so blanket numbers obscure real per-model differences. Finally, this report deliberately did not consult one adjacent body of work the user excluded, and it did not deeply cover a few related topics—quantization-induced degradation, RLHF-induced effects like sycophancy and mode collapse, and training-data contamination—which are real contributors to output quality but fall outside the context-and-tokens framing of the query. These are noted as gaps rather than treated.

RECOMMENDATIONS

For practitioners building on LLMs today, the evidence supports a concrete and prioritized playbook. **First, measure effective context, not claimed context.** Before trusting a model with long inputs, benchmark it on representative tasks at increasing lengths and identify where accuracy crosses below an acceptable threshold; treat anything beyond that "effective length" as a zone where critical facts may be

silently dropped [9][10]. As a starting heuristic pending your own measurements, keep working contexts under ~50K tokens, expect quality dips from 50–200K, and avoid relying on retrieval accuracy beyond ~200K [25].

Second, engineer the context as a budget. Curate aggressively toward the smallest high-signal set [6]; place the most important material at the beginning and end of the prompt and use a reranker to keep low-relevance noise out entirely [21][14]; enable compaction at explicit boundaries for long sessions, preserving decisions and open problems while clearing bulky tool outputs [19][28][33]; isolate independent subtasks in sub-agents with their own clean windows [6][8]; and offload durable state to external files the agent can re-read [8][6]. **Third, specify fully and prompt with economy.** Prefer a single complete instruction over a drip-feed of underspecified turns—or consolidate the conversation when it drifts [22][23]. For reasoning models specifically, strip chain-of-thought scaffolding and few-shot padding, state the problem clearly, keep instructions under roughly 3,000 tokens, and ensure the token cap leaves room for the hidden reasoning budget [26][32].

Fourth, allocate output tokens adaptively, not maximally. Do not assume "think longer" helps; tune reasoning effort to problem difficulty, stop at moderate budgets, and consider parallel sampling with majority vote over a single very long trace [15][16][17]. **Fifth, for long-horizon agents, segment and verify.** Break long tasks into bounded segments with intermediate checkpoints, add closed-loop verification (validators, checkers, or lookahead) so errors are caught before they compound, and retain a recovery mechanism rather than executing every step in irreversible isolation [12][29][31]. **Sixth, diagnose at the right layer.** Repetitive or incoherent *text* is a decoding-parameter problem (adjust temperature/top-p/penalties) [5]; fluent-but-wrong-or-forgetful output is a context/attention problem (apply the curation and placement fixes above) [3][18]. As a research-and-tooling priority, the field would benefit most from a standardized benchmark that scores mitigation strategies head-to-head and from production telemetry that tracks effective-length and compaction-thrashing in the wild.

METHODOLOGY APPENDIX

This report was produced with a structured deep-research pipeline (deep mode, eight phases: scope, plan, retrieve, triangulate, outline refinement, synthesize, critique, package). All source identity and evidence were persisted to disk under the run directory as append-only artifacts: `sources.jsonl` (33 sources with stable sha256-based identifiers), `evidence.jsonl` (verbatim evidence spans tied to source IDs), and `run_manifest.json` (query, mode, timestamps). Display citation numbers [N] are derived from registration order and map deterministically to the stable IDs.

Retrieval was conducted via parallel web search across eight pre-decomposed angles: context rot and long-context degradation; the lost-in-the-middle positional effect; multi-turn/conversational degradation; decoding-level degeneration; context-engineering mitigations; the failure-mode taxonomy; rigorous long-context benchmarks; autoregressive error accumulation; verbose-prompt and overthinking effects; and retrieval/architecture/decoding interventions. Searches returned both the primary literature (peer-reviewed

papers and preprints on arXiv/PMLR/OpenReview) and authoritative secondary sources (vendor engineering documentation and well-regarded practitioner analyses). Source types span academic (16), industry/vendor (9), and practitioner/blog (8), providing perspective diversity across researchers, model vendors, and builders.

Triangulation prioritized claims supported by multiple independent sources. The central thesis—that quality degrades with input length before the window is full—is independently established by an industry 18-model sweep [1], an academic synthetic-task benchmark [9], and an academic association-based benchmark [10]. The lost-in-the-middle effect [3] is corroborated by mechanistic work on attention sinks [13] and applied RAG studies [14][21]. The conservation-of-attention mechanism is stated consistently by a model vendor [6], an enterprise vendor [18], and an independent platform [20]. Where claims rested on a single source, vendor self-interest, or anecdote (e.g., the Pokémon-agent distraction ceiling [7]), this is flagged in-text and in the Limitations section.

Anti-hallucination controls: every factual claim is cited inline to a specific registered source; direct quotations are reproduced verbatim from retrieved page text and stored in the evidence ledger; synthesis and inference are explicitly marked as such; and no citation was created without a corresponding registered source. Topics outside the query's context-and-tokens framing (quantization, RLHF/sycophancy, data contamination) were deliberately scoped out and noted as gaps rather than covered superficially. Per the user's explicit instruction, no pre-existing research in the excluded directory was consulted at any phase.

BIBLIOGRAPHY

[1] Hong et al. (2025). [Context Rot: How Increasing Input Tokens Impacts LLM Performance](https://www.trychroma.com/research/context-rot). Chroma Research. [2] Morph (2025). [Context Rot: Why LLMs Degrade as Context Grows (Complete Guide)](https://www.morphllm.com/context-rot). [3] Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni & Liang (2023). [Lost in the Middle: How Language Models Use Long Contexts](https://arxiv.org/abs/2307.03172). arXiv:2307.03172 (TACL 2024). [4] Laban, Hayashi, Zhou & Neville (2025). [LLMs Get Lost In Multi-Turn Conversation](https://arxiv.org/abs/2505.06120). arXiv:2505.06120 (ICLR 2026). [5] Holtzman, Buys, Du, Forbes & Choi (2019). [The Curious Case of Neural Text Degeneration](https://arxiv.org/abs/1904.09751). arXiv:1904.09751 (ICLR 2020). [6] Anthropic (2025). [Effective context engineering for AI agents](https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents). Anthropic Engineering. [7] Breunig (2025). [How Long Contexts Fail](https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html). [8] Breunig (2025). [How to Fix Your Context](https://www.dbreunig.com/2025/06/26/how-to-fix-your-context.html). [9] Hsieh, Sun, Kriman, Acharya, Rekesh, Jia & Ginsburg (2024). [RULER:

What's the Real Context Size of Your Long-Context Language Models?](https://arxiv.org/abs/2404.06654). arXiv:2404.06654. [10] Modarressi, Deilamsalehy, Dernoncourt, Bui, Rossi, Yoon & Schuetze (2025). [NoLiMa: Long-Context Evaluation Beyond Literal Matching](https://proceedings.mlr.press/v267/modarressi25a.html). PMLR v267 (ICML 2025). [11] Sinha, Sarkar & Sengupta (2025). [The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs](https://arxiv.org/abs/2509.09677). arXiv:2509.09677. [12] (2026). [Intrinsic Stability Limits of Autoregressive Reasoning: Structural Consequences for Long-Horizon Execution](https://arxiv.org/abs/2602.06413). arXiv:2602.06413. [13] Xiao, Tian, Chen, Han & Lewis (2023). [Efficient Streaming Language Models with Attention Sinks](https://arxiv.org/abs/2309.17453). arXiv:2309.17453 (ICLR 2024). [14] (2024). [Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG](https://arxiv.org/abs/2410.05983). arXiv:2410.05983. [15] (2026). [When More Thinking Hurts: Overthinking in LLM Test-Time Compute Scaling](https://www.arxiv.org/abs/2604.10739). arXiv:2604.10739. [16] (2025). [Does Thinking More always Help? Understanding Test-Time Scaling in Reasoning Models](https://arxiv.org/abs/2506.04210). arXiv:2506.04210. [17] (2025). [When More Tokens Hurt: Saturation Effects in Test-Time Compute Scaling](https://openreview.net/pdf?id=RoTJh22LGx). OpenReview. [18] Salesforce (2025). [What Is Context Rot?](https://www.salesforce.com/artificial-intelligence/ai-context/context-rot/). [19] Anthropic (2026). [Compaction - Claude API Docs](https://platform.claude.com/docs/en/build-with-claude/compaction). [20] deepset (2025). [Context Engineering for Agentic Systems](https://haystack.deepset.ai/blog/context-engineering). Haystack Blog. [21] Pinecone (2024). [Rerankers and Two-Stage Retrieval](https://www.pinecone.io/learn/series/rag/rerankers/). [22] PromptHub (2025). [Why LLMs Fail in Multi-Turn Conversations (And How to Fix It)](https://www.prompthub.us/blog/why-llms-fail-in-multi-turn-conversations-and-how-to-fix-it). [23] (2026). [Intent Mismatch Causes LLMs to Get Lost in Multi-Turn Conversation](https://www.arxiv.org/abs/2602.07338). arXiv:2602.07338. [24] (2026). [The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break](https://arxiv.org/abs/2604.11978). arXiv:2604.11978. [25] AI Expert OÜ (2025). [Context engineering: managing 1M-token windows without context rot](https://aiexpert.ee/en/articles/context-engineering). [26] (2025). [Prompting reasoning models](https://llmbestpractices.com/prompt-engineering/reasoning-model-prompting). llmbestpractices. [27] Breunig (2025). [Working with Contexts](https://www.oreilly.com/radar/working-with-contexts/). O'Reilly Radar. [28] Dibia (2025). [Context Engineering 101: How Agents Manage Context](https://newsletter.victordibia.com/p/context-engineering-101-how-agents). [29] (2026). [LEAD: Breaking the No-Recovery Bottleneck in Long-Horizon Reasoning](https://www.arxiv.org/abs/2603.06870). arXiv:2603.06870. [30] Chen et al. (2025). [Do NOT Think That Much for 2+3=? On the Overthinking of Long Reasoning Models](https://proceedings.mlr.press/v267/chen25bx.html). PMLR v267 (ICML 2025). [31] (2026). [Why Reasoning Fails to Plan: A Planning-Centric Analysis of Long-Horizon Decision Making in LLM Agents](https://arxiv.org/abs/2601.22311). arXiv:2601.22311. [32] TruvaG3 (2025). [Effective Prompts Guide](https://docs.truvag3.dev/docs/building/EFFECTIVE_PROMPTS_GUIDE/). [33] (2025). [Context, Memory, and Compaction](https://github.com/DenisSergeevitch/agents-best-practices/blob/main/references/context-memory-compaction.md). agents-best-practices.